MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Cahier de recette

Programmation du Plakoto

Auteurs :
YA Hassrol
GHANIME Joseph
BENNAMANE Mahdi
REN Ziyi

Encadrant :
BOUZY Bruno

Groupe : L2N1

Table des matières

1 Introduction **3**
1.1 Objectifs et méthodes 3
1.2 Documents de référence 3

2 Concepts de base **3**
2.1 Règles du jeu Plakoto 3
2.2 Règles de test dans le cahier de recette 3

3 Description de la fourniture **4**
3.1 Contenu du livrable 4
3.2 Documentation associée 4

4 Moyen d'essai et d'outils **4**
4.1 ***Outils de test*** 4
4.2 Procédures de test 4

5 Conformité aux spécifications générales **4**
5.1 ***Respect des exigences fonctionnelles*** 4
5.2 ***Performance et optimisation*** 5
5.3 ***Fiabilité et robustesse*** 5

6 Conformité aux spécifications fonctionnelles **5**
6.1 ***Blocage d'un pion seul (« trapping »)*** 5
6.1.1 **Objectif** 5
6.1.2 **Description** 5
6.1.3 **Contraintes et dépendances** 5
6.1.4 **Procédure de test** 5
6.1.5 **Résultats attendus** 6
6.2 ***Prime : Chaîne de blocages*** 6
6.2.1 **Objectif** 6
6.2.2 **Description** 6
6.2.3 **Contraintes et dépendances** 6
6.2.4 **Procédure de test** 6
6.2.5 **Résultats attendus** 6
6.3 ***Gestion des doublets*** 6
6.3.1 **Objectif** 6
6.3.2 **Description** 7
6.3.3 **Contraintes et dépendances** 7
6.3.4 **Procédure de test** 7
6.3.5 **Résultats attendus** 7

7 Conformité aux spécification d'interface **7**
7.1 ***Objectif*** 7
7.2 ***Critères de conformité*** 7
7.3 ***Tests de validation*** 8
7.3.1 **Interface graphique (Pygame)** 8
7.3.2 **Interface textuelle** 8

7.4 ***Conformité aux interactions utilisateur*** 8
7.5 ***Évaluation et ajustements*** 8

8 Conformité de la documentation **9**
8.1 ***Objectif*** 9
8.2 ***Vérification de l'exhaustivité*** 9
8.3 ***Clarté et structuration*** 9
8.4 ***Références et annexes*** 9
8.5 ***Types de documentation*** 9
8.6 ***Validation et maintenance*** 10

9 Annexe **10**
9.1 ***Règles du Plakoto (Référence aux règles du jeu)*** 10

10 Glossaire **11**

11 Références **12**

1 Introduction

Ce document est un cahier de recettes, dont le but est de décrire le contexte, et le détail du déroulement des étapes du projet. Ces étapes sont la soumission, la vérification et la validation du projet. Cela nous permettra de vérifier si les objectifs du projet ont été atteints. Nous allons donc lister chaque fonctionnalité du Plakoto, et les tests qu'elles doivent passer.

1.1 Objectifs et méthodes

L'objectif de ce projet est de créer un jeu fonctionnel de Plakoto. Les utilisateurs pourront lancer des dés, déplacer leur pions et jouer entre eux. De plus, les utilisateurs pourront jouer contre une IA utilisant l'approche Monte Carlo. Si nous disposons du temps nécessaire, ils pourront aussi jouer au Plakoto à l'aide d'une interface graphique.

1.2 Documents de référence

— Cahier des charges
— Plan de test
— Manuel d'utilisation

2 Concepts de base

2.1 Règles du jeu Plakoto

— **Déplacement des pions** : Chaque joueur déplace ses pions en fonction des résultats du lancer de dés, en respectant les règles spécifiques du Plakoto.
— **Blocage ("Trapping")** : Lorsqu'un pion atterrit sur une case occupée par un seul pion adverse, il l'immobilise en restant au-dessus.
— **Prime (chaîne de blocages)** : Un joueur qui bloque six cases consécutives empêche son adversaire d'avancer.
— **Utilisation des doublets** : Un double permet de jouer quatre fois le même nombre obtenu.
— **Conditions de victoire** : Un joueur gagne lorsqu'il a retiré tous ses pions du plateau.

2.2 Règles de test dans le cahier de recette

— Tous les tests doivent être reproductibles.
— Chaque test doit avoir un objectif clair et une procédure détaillée.
— Les résultats attendus doivent être définis afin de valider ou d'invalider le test.

3 Description de la fourniture

3.1 Contenu du livrable

Le projet final inclus :
- Une **application autonome** exécutable contenant le jeu Plakoto.
- Une **interface textuelle et graphique** permettant l'interaction des joueurs avec le jeu.
- Une **intelligence artificielle** jouant selon la méthode Monte Carlo.
- Un **système de sauvegarde et chargement de partie** (optionnel).

3.2 Documentation associée

Les documents fournis incluent :
- **Un manuel d'utilisation** décrivant comment jouer et utiliser l'application.
- **Une documentation technique** détaillant l'architecture du projet et les choix de développement.
- **Un glossaire** expliquant les termes spécifiques utilisés dans le projet.

4 Moyen d'essai et d'outils

Cette section décrit les outils et méthodes utilisés pour tester le jeu et valider les fonctionnalités.

4.1 Outils de test

- **Python (unittest, pytest)** : Utilisé pour tester les différentes fonctionnalités du jeu.
- **Pygame** : Permet de tester l'affichage de l'interface graphique.
- **Journalisation des actions** : Un fichier log enregistre les interactions des joueurs et les erreurs rencontrées.

4.2 Procédures de test

- **Tests unitaires** : Vérification des fonctions du jeu (déplacement, capture, IA).
- **Tests d'interface utilisateur** : Validation des commandes textuelles et graphiques.
- **Tests d'intégration** : Vérification de la communication entre les modules.

5 Conformité aux spécifications générales

5.1 Respect des exigences fonctionnelles

- Le jeu doit être jouable en **mode solo contre une IA** et en **mode multijoueur local**.
- Les règles du Plakoto doivent être fidèlement respectées.
- Les actions des joueurs doivent être prises en compte sans latence.

5.2 Performance et optimisation

— L'IA doit répondre en **moins de 5 secondes**.
— Le jeu doit fonctionner de manière fluide, sans ralentissements.

5.3 Fiabilité et robustesse

— Le jeu ne doit pas crasher, même en cas d'entrées utilisateur incorrectes.
— Les erreurs doivent être gérées avec des messages explicatifs pour l'utilisateur.

6 Conformité aux spécifications fonctionnelles

L'objectif de cette section est de valider que les fonctionnalités essentielles du jeu Plakoto ont été implémentées conformément aux règles traditionnelles et aux exigences définies dans le cahier des charges. Les tests ci-dessous couvrent les principaux mécanismes de jeu, à savoir le blocage d'un pion isolé, la formation de chaînes de blocages et la gestion des doublets. Chaque scénario est conçu pour être reproductible et permet de détecter rapidement toute déviation du comportement attendu.

6.1 Blocage d'un pion seul (« trapping »)

6.1.1 Objectif

Vérifier que lorsqu'un pion isolé se trouve sur une case, il peut être immobilisé par un pion adverse se positionnant sur cette même case.

6.1.2 Description

Ce test simule la situation dans laquelle un joueur tente de bloquer un pion isolé de l'adversaire. Lorsqu'un pion adverse se déplace sur une case occupée par un pion seul, ce dernier doit être bloqué et ne doit pas pouvoir être déplacé tant que le blocage persiste.

6.1.3 Contraintes et dépendances

— L'environnement de test doit permettre un contrôle précis des positions sur le plateau.
— La fonctionnalité de déplacement doit être pleinement opérationnelle.
— L'interface utilisateur doit être en mesure d'afficher un message d'erreur clair lorsqu'une action interdite est tentée.

6.1.4 Procédure de test

— Placer un pion isolé sur une case spécifique du plateau.
— Déplacer un pion adverse afin de se positionner sur cette même case.
— Tenter de déplacer le pion désormais bloqué.
— Vérifier que le système empêche ce déplacement et affiche un message d'erreur explicatif.

6.1.5 Résultats attendus

— Le pion bloqué ne doit pas être déplaçable tant que le blocage est en place.
— L'interface affiche une notification indiquant clairement que le déplacement est impossible en raison du blocage adverse.

6.2 Prime : Chaîne de blocages

6.2.1 Objectif

S'assurer qu'une chaîne de six pions consécutifs empêche l'avancée de l'adversaire sur ces cases.

6.2.2 Description

Ce test vérifie que lorsque six cases adjacentes sont occupées par les pions d'un même joueur, l'adversaire est totalement bloqué sur ces points. Le système doit refuser toute tentative de déplacement vers ces cases et signaler l'impossibilité d'avancer.

6.2.3 Contraintes et dépendances

— Il est nécessaire de configurer le plateau afin de disposer six pions consécutifs appartenant au même joueur.
— La fonctionnalité de vérification des déplacements doit détecter automatiquement les zones bloquées.

6.2.4 Procédure de test

— Installer six pions consécutifs sur le plateau, tous appartenant au même joueur.
— Faire en sorte que l'adversaire tente de déplacer l'un de ses pions vers l'une de ces cases bloquées.
— Observer la réponse du système et vérifier que le déplacement est refusé.

6.2.5 Résultats attendus

— Le système empêche l'adversaire d'accéder aux cases composant la chaîne de blocages.
— Un message d'erreur ou une alerte visuelle informe l'utilisateur que le passage est impossible en raison de la prime active.
— En retirant l'un des pions de la chaîne, le blocage doit être levé et l'adversaire doit être autorisé à jouer.

6.3 Gestion des doublets

6.3.1 Objectif

Valider que lorsqu'un joueur obtient un doublet lors du lancer de dés, il peut effectuer quatre mouvements correspondant à la valeur obtenue.

6.3.2 Description

Ce scénario de test s'assure que la règle des doublets est correctement appliquée. Lorsqu'un double (par exemple, 2-2) est obtenu, le joueur doit pouvoir réaliser quatre déplacements consécutifs avec la valeur indiquée. Le système doit mettre à jour le plateau après chaque mouvement et vérifier la validité de chaque action.

6.3.3 Contraintes et dépendances

— Le module de lancer de dés doit être capable de générer un double de manière contrôlée pour le test.
— Les fonctions de déplacement doivent accepter et traiter quatre mouvements successifs basés sur la même valeur.

6.3.4 Procédure de test

— Effectuer un lancer de dés jusqu'à obtenir un double (exemple : 2-2) .
— Réaliser successivement quatre déplacements utilisant la valeur obtenue.
— Vérifier que, pour chaque mouvement, l'interface met à jour correctement les positions des pions sur le plateau.
— Confirmer que toutes les règles associées au doublet sont respectées.

6.3.5 Résultats attendus

— Le joueur doit être autorisé à réaliser quatre déplacements valides à partir du doublet.
— Les positions des pions doivent être mises à jour de manière cohérente après chaque mouvement.
— L'interface doit fournir un retour visuel confirmant l'application correcte de la règle des doublets.

7 Conformité aux spécification d'interface

7.1 Objectif

L'objectif de cette section est de **vérifier que l'interface utilisateur** respecte les exigences définies dans le cahier des charges. Cette validation concerne aussi bien **l'interface graphique (Pygame)** que **l'interface textuelle**. L'expérience utilisateur doit être intuitive, fluide et répondre aux critères d'accessibilité et d'ergonomie.

7.2 Critères de conformité

L'interface doit répondre aux critères suivants :
— **Conformité aux maquettes** définies dans le cahier des charges.
— **Cohérence visuelle** (positionnement des éléments, couleurs, typographie).
— **Réactivité et fluidité** (pas de latence dans les animations ou les actions utilisateur).
— **Accessibilité** (lisibilité des textes, taille des boutons, compatibilité avec différentes résolutions d'écran).
— **Ergonomie** (facilité de navigation, compréhension des icônes et des interactions).

— **Compatibilité** avec différents systèmes d'exploitation et environnements (Windows, Linux, MacOS).

7.3 Tests de validation

Les tests suivants permettent de vérifier la conformité de l'interface utilisateur :

7.3.1 Interface graphique (Pygame)

ID	Test	Attendu	Résultat
T6	Affichage correct du plateau	Le plateau s'affiche entièrement	
T7	Déplacement des pions par clic	Le pion sélectionné se déplace	
T8	Animation des dés	Affichage fluide des dés	
T9	Feedback visuel (confirmation, erreurs)	Messages et animations visibles	

7.3.2 Interface textuelle

ID	Test	Attendu	Résultat
T10	Commande help	Liste des commandes affichée	
T11	Déplacement d'un pion avec commande	Le pion se déplace	
T12	Message d'erreur pour une commande invalide	Message d'erreur affiché	

7.4 Conformité aux interactions utilisateur

Les interactions doivent être conformes aux attentes définies dans les spécifications :
— **Navigation intuitive** : L'utilisateur doit comprendre rapidement comment interagir avec le jeu.
— **Retours visuels immédiats** en cas d'action réussie ou d'erreur.
— **Aucune erreur bloquante** ne doit empêcher le joueur d'effectuer une action prévue par les règles.

7.5 Évaluation et ajustements

— **Retour des utilisateurs** : Des tests avec des joueurs seront effectués pour valider l'intuitivité de l'interface.
— **Corrections et améliorations** basées sur les résultats des tests et les retours d'expérience.
— **Mise à jour continue** pour assurer l'optimisation et l'amélioration des interfaces.

8 Conformité de la documentation

8.1 Objectif

L'objectif de cette section est de garantir que la documentation fournie avec le projet est **exhaustive**, **bien structurée** et **compréhensible** aussi bien par les **utilisateurs finaux** que par les **développeurs** souhaitant modifier ou améliorer le jeu. Elle doit assurer la traçabilité des informations et permettre une utilisation et une maintenance efficace du logiciel.

8.2 Vérification de l'exhaustivité

La documentation doit couvrir **tous les aspects** du projet, notamment :
- Une **présentation générale** du projet et de ses objectifs.
- Une **description détaillée** des fonctionnalités implémentées.
- Une explication des **règles du jeu** et leur adaptation au format numérique.
- Une présentation des **choix techniques** et des **algorithmes utilisés** (notamment l'IA Monte Carlo).
- Une section dédiée aux **erreurs potentielles** et aux solutions associées.
- Un résumé des **tests effectués** pour garantir la conformité du jeu aux spécifications.
- Une liste des **dépendances et prérequis techniques** nécessaires à l'exécution du logiciel.

8.3 Clarté et structuration

Pour être efficace, la documentation doit respecter les critères suivants :
- **Lisibilité** : Rédaction claire et concise, avec un langage adapté au public cible.
- **Organisation** : Structure logique avec des **titres bien définis** et une **hiérarchisation des sections**.
- **Accessibilité** : Un **sommaire détaillé** doit être inclus pour faciliter la navigation.
- **Glossaire** : Tous les **termes techniques spécifiques** au projet doivent être définis afin d'éviter toute ambiguïté.
- **Illustrations et schémas** : Des **captures d'écran**, **diagrammes** ou **exemples de code** doivent être intégrés pour faciliter la compréhension.

8.4 Références et annexes

- **Références** : Toutes les sources utilisées pour le développement du projet doivent être correctement citées (ex. documentation Python, articles techniques sur Monte Carlo, règles du Plakoto, etc.).
- **Annexes** : Des annexes doivent être incluses si nécessaire pour fournir des **explications supplémentaires** (ex. schémas explicatifs, captures d'écran du jeu).
- **Documentation externe** : Toute ressource externe utile à la compréhension ou à l'utilisation du jeu doit être mentionnée.

8.5 Types de documentation

La documentation doit être divisée en **deux parties distinctes** :

1. **Documentation technique**

— Contient les détails liés à **l'architecture du projet**, aux **choix technologiques**, aux **modules utilisés** et aux **instructions** pour les développeurs souhaitant modifier le code.
— Décrit le **plan de test**, le **cahier de recette**, et les procédures de validation mises en place.

2. **Documentation utilisateur**
   — Décrit l'**utilisation du jeu**, les **commandes disponibles**, les **scénarios de jeu**, et les **solutions aux problèmes courants**.
   — Fournit un **manuel d'installation** avec des **instructions détaillées** sur la configuration du jeu.

8.6 Validation et maintenance

— La documentation doit être **régulièrement mise à jour** en fonction des évolutions du projet.
— Une **vérification finale** doit être effectuée pour garantir la **cohérence** des informations fournies avec la **version actuelle du jeu**.
— Un **retour d'expérience des utilisateurs** peut être pris en compte pour **améliorer la clarté et la pertinence du document**.
— La documentation doit être évaluée par **les membres de l'équipe** pour s'assurer qu'elle répond aux besoins des développeurs et des joueurs.

9 Annexe

9.1 Règles du Plakoto *(Référence aux règles du jeu)*

Rappel des règles officielles du Plakoto Les règles du Plakoto utilisées pour la validation des tests respectent les standards du jeu traditionnel et sont définies comme suit :

— Chaque joueur dispose de **15 pions**, qui doivent être déplacés selon un mouvement circulaire autour du plateau.
— Le jeu utilise **deux dés**, et le joueur doit déplacer ses pions en fonction des valeurs obtenues.
— Un pion **ne peut avancer** que si la case d'arrivée est libre ou occupée par un de ses propres pions.
— Contrairement au backgammon classique, il n'y a **pas de barre** : un pion capturé est bloqué et **ne peut plus bouger** tant que l'adversaire contrôle la case.

Explication des mécanismes clés **Déplacement des pions**

— Les joueurs avancent leurs pions dans des directions opposées.
— Un pion peut être déplacé selon la valeur obtenue sur l'un des dés, ou être déplacé en deux étapes avec les deux dés combinés.
— Lorsqu'un joueur obtient un **double** (ex. 3-3), il peut utiliser chaque valeur **quatre fois** pour déplacer ses pions.

Conditions de blocage (*Prime*)

— Lorsqu'un joueur empile **deux pions ou plus** sur une case, cette case devient une **barrière imprenable**.

— Si **six cases consécutives** sont bloquées par un joueur, l'adversaire ne pourra plus avancer et risque d'être **complètement piégé**.

Utilisation des doubles dés

— Un double permet de **jouer quatre fois** la valeur obtenue.
— Cela favorise des stratégies défensives (*former des blocs*) et offensives (*avancer rapidement plusieurs pions*).

Conditions de victoire

— Un joueur gagne lorsqu'il **déplace tous ses pions sur la dernière case et qu'ils sont bloqués définitivement**.
— Si un joueur ne peut plus effectuer de **mouvement légal** à cause d'un blocage total, il **perd immédiatement**.
— En cas de blocage mutuel, la partie est déclarée **nulle**.

Schéma explicatif Une **image du plateau** illustrant les principales règles et stratégies (mouvements, blocages, conditions de victoire) peut être ajoutée pour une meilleure compréhension.

10 Glossaire

1. **Backgammon**
   Jeu de plateau ancien qui combine stratégie et hasard. Le backgammon constitue la base sur laquelle s'appuie le Plakoto et permet de mieux comprendre l'origine des règles du jeu.
2. **Bug**
   Erreur ou comportement inattendu dans le jeu empêchant une fonctionnalité de fonctionner correctement. L'identification et la correction des bugs sont des étapes essentielles du processus de recette.
3. **Compatibilité multi-plateforme**
   Vérification du bon fonctionnement du jeu sur différents systèmes d'exploitation (Windows, macOS, Linux) afin de garantir une expérience utilisateur homogène.
4. **Critères d'acceptabilité**
   Conditions minimales à remplir pour considérer qu'une fonctionnalité est conforme et validée. Ces critères sont définis en amont et servent de référence lors des tests.
5. **Détection d'anomalies**
   Processus permettant d'identifier les incohérences ou erreurs lors des tests. Il s'agit d'une étape clé pour améliorer la robustesse du jeu avant sa mise en production.
6. **IA (Intelligence Artificielle)**
   Dans le cadre du projet, l'IA désigne le module qui permet de jouer contre l'ordinateur. Elle est conçue pour analyser le plateau, anticiper les coups et prendre des décisions optimisées en utilisant l'approche Monte Carlo. L'IA doit répondre en moins de cinq secondes pour garantir une expérience fluide.
7. **Interface utilisateur**
   Terme qui englobe l'ensemble des éléments graphiques et textuels permettant à l'utilisateur d'interagir avec le jeu. Que ce soit via une interface textuelle ou graphique, l'interface doit être claire, intuitive et facile à prendre en main.

8. **Méthode Monte Carlo**
   Technique de simulation probabiliste utilisée pour explorer et évaluer de multiples scénarios de jeu. Cette méthode aide l'intelligence artificielle à choisir le coup le plus efficace en se basant sur des résultats statistiques issus de nombreuses simulations.
9. **Pygame**
   Bibliothèque Python dédiée au développement de jeux vidéo. Elle facilite la création de l'interface graphique du jeu, la gestion des animations, des interactions utilisateur et l'affichage des différents éléments du plateau et des pions.
10. **Plakoto**
    Variante du backgammon, le Plakoto se distingue par ses règles spécifiques, notamment la possibilité de bloquer un pion adverse en recouvrant sa case. Ce terme est central pour le projet, puisque le jeu développé doit respecter fidèlement les mécanismes traditionnels du Plakoto.
11. **Rapport de test**
    Document contenant les résultats des tests effectués, listant les réussites et les échecs, ainsi que les éventuelles corrections à apporter avant la validation finale.
12. **Régression**
    Phénomène où une modification du code entraîne l'apparition d'un bug dans une fonctionnalité précédemment validée. Des **tests de non-régression** sont effectués pour éviter cela.
13. **Scénario de test**
    Description détaillée d'une séquence d'actions à exécuter pour tester une fonctionnalité spécifique. Chaque scénario comprend une **identification**, une **description**, les **contraintes**, les **dépendances** et la **procédure de test**.
14. **Test utilisateur**
    Phase de validation où des utilisateurs réels testent l'application afin de vérifier l'ergonomie, la facilité d'utilisation et la satisfaction générale.
15. **Test de charge**
    Évaluation des performances du jeu lorsqu'un grand nombre d'opérations sont effectuées simultanément, notamment en mode multijoueur local.
16. **Validation des tests**
    Processus permettant de s'assurer que les tests effectués respectent les critères d'acceptabilité définis dans le Cahier de Charges. Il inclut la vérification des résultats attendus et la correction des éventuelles anomalies.

11 Références

Documentation technique et outils de test : Références aux documentations officielles de Python, Pygame, Unittest, et pytest, qui ont été utilisées pour la mise en œuvre et la validation technique du projet.

Articles et ressources en ligne : Des ressources sur la validation logicielle, les meilleures pratiques de tests unitaires et d'intégration, ainsi que le développement de jeux en Python.

Ouvrages de référence : Livres clés qui ont contribué à la structuration et à l'optimisation des tests et du développement du projet.